VandLabs Automobile Engine

Integration status, what we still need, where to get it, and the launch order
Prepared 25 October 2026 · branch `claude/modest-gauss-7s6y71`

One-paragraph summary. The code for all five pieces is built, linked together, and tested on a developer machine. **Nothing is live yet.** To go live you need to create a handful of accounts and credentials (Cloudflare, Neon, AWS Cognito, Google, optionally Resend), load the dealership's real data (dealership details, staff, stock), and sign a licence for any outside market-data feed. Section 4 and 5 list every item and where it comes from. Section 6 is the order to do it in.

1. The five pieces

Piece	What it is	Where it runs
Public website apps/web	The dealership's showroom for buyers: inventory search, enquiries, test drives, the `/dealer` product page.	Port 3000 / Cloudflare (behind the gateway)
Command Center apps/command-center	Staff back office under `/command`: leads, inventory, capital, sourcing, dealer API keys.	Port 3001 / Cloudflare (behind the gateway)
Platform apps/platform	VandLabs' own admin console under `/platform`: staff, subscriptions, data-source approvals.	Port 3002 / Cloudflare (behind the gateway)
Automobile Intelligence apps/intelligence	The dealer product: pricing intelligence, advertising studio, capital and profit, alerts, reports, market radar.	Port 3003 / its own Cloudflare Worker (or Vercel) on its own address
Desktop app apps/desktop	Windows / macOS client (Tauri) for the dealer's daily work: leads, follow-ups, stock, capital.	Installed on staff computers

All of them read and write the same Neon (PostgreSQL) database. Every dealership's rows are separated by row-level security, so one dealer can never see another's data.

2. How they are connected now

Link	Status	How
Same database and tenant rules	**Done**	Intelligence maps onto the shared tables (vehicles, leads, tasks, sales, staff). Command Center and Intelligence show the same stock and leads.
Website → Intelligence	**Done**	The `/dealer` page "Open dealer workspace" button points to Intelligence when `INTELLIGENCE_URL` is set.

Command Center → Intelligence	**Done**	Sidebar link "Open Automobile Intelligence" for signed-in staff (`INTELLIGENCE_URL`).
Desktop → Intelligence	**Done**	The desktop app learns the Intelligence address from the workspace and shows an "Open Intelligence" button that opens the system browser. The address must be a plain HTTPS origin. Rust unit logic was tested separately; the full desktop build could not be run in this environment (needs GTK libraries), so build it once in CI before release.
Google single sign-in	**Built**	Cognito is the one identity provider and Google is added to it. One verified email = one staff account across Command Center and Intelligence. Needs your Google and AWS setup (section 4). Passwords still work in Intelligence.
`pnpm demo` runs all four web apps	**Done**	Starts 3000, 3001, 3002 and 3003 and waits until all four answer. Intelligence's sign-in needs `DATABASE_URL` and `SESSION_SECRET` locally.
Cloudflare Worker for Intelligence	**Done**	Builds as its own Worker on its own hostname (not behind the gateway, because its cookies and routes need their own address). Only deploys once `CF_INTELLIGENCE_ORIGIN` is set.
Platform → Google sign-in	**Not included**	Platform admins still use Cognito directly. Google does not grant platform-admin access (deliberate).
Desktop app Google sign-in	**Not included**	Stays on Cognito sign-in with MFA (deliberate).

3. What is and is not live

Be clear with customers and investors: all of the above is tested locally (lint, type checks, 40 security tests, 35 database tests, Intelligence production tests, Cloudflare configuration tests, and an infrastructure template test). It has **not** been run against real Google, real AWS Cognito, or the live Cloudflare deployment. The earlier repository status files say the same: staging acceptance and production cutover are still pending.

- **Verified here:** the Google login handshake logic (against a stand-in for Cognito), account linking rules, invitation by Google, disabled-account blocking, the Cloudflare Worker bundle build, the demo launcher with all four apps answering.
- **Not verified anywhere yet:** a real Google click-through, a real Cognito token, the live Worker, MFA enrolment, real inventory, real notifications.
- **Known issue:** the Next.js version pinned in the project (15.5.27) is now past its support window. The Cloudflare build tool refuses to run unless forced. This affects *all* apps, not only the new one. Plan an upgrade before relying on automated Cloudflare builds.
- Desktop installers have no code-signing set up, so Windows/macOS will show "unknown publisher" warnings until certificates are bought and added.

4. What we need to create, and where to get it

Rule: secrets (passwords, tokens, API keys) never go in chat, in git, or in screenshots. They go into the GitHub environment secret store or the host's secret settings.

Need	What exactly	Where / how to get it
Cloudflare account	Account ID, a **paid** Workers plan, R2 storage switched on, a scoped API token (Workers Scripts edit, Workers AI, R2, account read).	dash.cloudflare.com. R2 activation and billing can only be done in the dashboard. Create the token under My Profile → API Tokens. Account ID is on the dashboard overview. (A Cloudflare account already exists in the project notes; R2 and billing were still blocked.)
Domain names	One for the main site gateway, one for Intelligence (for example `intelligence.yourdomain`).	Any registrar, or Cloudflare Registrar. Until then a free `*.workers.dev` address works for staging.
Neon database	Pooled URL for the restricted `vandlabs_runtime` role (apps); direct admin URL (migrations only).	console.neon.tech → the existing project. Apply migrations with `pnpm db:migrate` (now includes 0010 Intelligence and 0011 single sign-in). Take a backup branch first. Never put the admin URL on a web deployment.
AWS account (Cognito)	Region ap-south-1 (Mumbai). User pool with MFA required, no self-signup. Output values: user-pool ID, hosted domain, web client ID, desktop client ID, Intelligence client ID.	aws.amazon.com. Upload `infra/aws/templates/staff-identity-staging.json` in CloudFormation (steps in `docs/STAFF_IDENTITY_SETUP.md`). The repo notes say Cognito's free tier covers 10,000 monthly active users, but check billing.
Google sign-in	An OAuth client ID and client secret.	console.cloud.google.com → APIs & Services → Credentials → Create OAuth client (Web). Redirect URI: `https://<cognito-domain>/oauth2/idpresponse`. Give the secret to the CloudFormation stack as the hidden `GoogleClientSecret` parameter.
App secrets	`SESSION_SECRET` (apps), `INTELLIGENCE_SESSION_SECRET` (Intelligence, different), `DEALER_AI_SERVICE_TOKEN`.	Generate yourself: `openssl rand -base64 32` (Dealer AI token: 32 random bytes in base64url, 43 characters).
Email notifications (optional)	Resend API key, verified sender address, recipient.	resend.com. Verify your sending domain with the DNS records they give you. Notifications stay off until this is done.
GitHub	Environment `cloudflare-staging` with variables and secrets.	Repository → Settings → Environments. Variable list is in `docs/LIVE_READINESS.md`.
Desktop signing (later)	Apple Developer ID and a Windows code-signing certificate.	developer.apple.com (paid programme); a certificate authority for Windows. Not needed to test internally.

Environment settings added by this work

Setting	Goes on	Value
`INTELLIGENCE_URL`	Website, Command Center	HTTPS origin of Intelligence (for example `https://intelligence.yourdomain`). Local demo sets `http://127.0.0.1:3003` automatically.
`APP_ORIGIN`	Intelligence	Its own HTTPS origin (now used at runtime for Google sign-in).
`COGNITO_DOMAIN`	Intelligence	Cognito hosted sign-in domain.
`INTELLIGENCE_COGNITO_CLIENT_ID`	Intelligence	New stack output `CognitoIntelligenceClientId`.
`CF_INTELLIGENCE_ORIGIN`	Cloudflare configure step	The Intelligence address; without it the Worker is not deployed.
`DATABASE_URL`, `INTELLIGENCE_SESSION_SECRET`	Intelligence Worker	Pooled runtime URL; 32+ random characters.

5. What data we need, and who supplies it

Nothing in the product is faked in production. A new workspace starts empty, and every figure is calculated from real records. Items marked "setup required" in the app stay locked until the data or contract below exists.

Data	What it contains	Who supplies it / how it gets in
Dealership profile	Legal organisation name, dealership display name, each branch (name, city, state, phone), brand text, website address (canonical base).	The dealer. Loaded once by an administrator with `pnpm intelligence:provision` (Intelligence) and `pnpm db:provision-tenant` (shared tenant file). The tenant file's website address must equal the public site address.
Owner account	Owner's name and work email.	The dealer. The provisioning command prints a one-time setup link; the owner sets a password or signs in with Google using that same email.
Staff list	Name, work email, role (owner, manager, sales, inventory, marketing, finance, viewer), branches.	The owner invites staff in Intelligence → Settings → Team. People sign in with Google using the invited email. For Command Center roles see `docs/STAFF_ROLES.md`.
Inventory	Stock ID, make, model, variant, year, price (INR), mileage, fuel, transmission, ownership, body type, condition, colours, photos, finance/exchange flags.	The dealer's stock sheet. Enter one by one, or upload the CSV/Excel template in Command Center → Inventory (up to 200 rows per file; preview before import). Photos go to private storage (Cloudflare R2).

Costs and sales	Purchase price, preparation, transport, acquisition date, reference; confirmed sales (price and time).	The dealer's books. Needed for capital, profit and dead-stock alerts. Costs are never guessed: unrecorded costs show as unknown.
Leads and customers	Name, phone, source, interest.	Arrive automatically from website enquiries. Older leads can be entered by staff. Hosted demo mode is read-only.
Licensed market / supplier data	Listings with asking prices and timestamps, per source, with an agreement reference and expiry.	A **contract with a data provider** or a supplier who pushes stock to you. Website access is not a licence: marketplaces such as CarDekho, CarWale, Cars24 and OLX forbid scraping in their terms, so none is connected. Once licensed: import JSON in Command Center → Data sources, run a server-side feed sync, or let a supplier push through the ingest API with a key. A VandLabs admin then marks the source "dealer use allowed" in Platform → Commerce.
Advertising accounts	Connection to the ad platforms the dealer uses, plus a spend cap.	The dealer's own ad accounts, enabled per workspace by VandLabs once API credentials from each platform exist. Until then the Advertising Studio says "Setup required".
Fuel prices	Timestamped local petrol/diesel prices.	From a licensed source or entered by the dealer; used in ownership-cost estimates.
Subscription plans	Starter, Growth, Scale limits are in the code; prices are not.	You decide prices. Automated billing is not built; plans are provisioned by a VandLabs admin against a contract or invoice reference.

6. Launch order (staging first, then production)

1. **Accounts:** finish Cloudflare (paid Workers plan, R2 on, API token), AWS and Google (section 4). Choose the website and Intelligence addresses.
2. **Database:** make a Neon backup branch, then run `pnpm db:migrate` on staging (applies 0010 and 0011 plus any pending). Check the runtime role has no bypass of row-level security.
3. **Identity:** create the Cognito stack with the Intelligence address and Google client ID; enter the Google secret as the hidden parameter. Copy the outputs into GitHub environment secrets. Create the first platform administrator in Cognito and enrol their authenticator app.
4. **Configure and deploy:** run the GitHub workflow "Cloudflare build and deploy" with `deploy=false` (checks only), then `deploy=true`. Set `CF_INTELLIGENCE_ORIGIN` and `INTELLIGENCE_URL` so Intelligence is deployed and linked.
5. **Provision the first dealership:** run the tenant and workspace provisioning commands with the real details; send the owner the one-time link.
6. **Load data:** staff invites, then inventory, then costs. Leave market data and ad accounts "Setup required" until contracts exist.
7. **Acceptance on staging:** sign in with Google in Intelligence, open Command Center from the sidebar (no second password prompt), try the desktop app, submit a website enquiry and see it as a lead, disable a test user and confirm they are locked out, check security headers.

8. **Production:** repeat with a fresh Neon branch backup, then switch the website to indexable only after the owner approves. Keep the existing hosting until acceptance passes; rollback is by Worker version.

7. Decisions still open

- **Retire passwords in Intelligence?** Currently both Google and passwords work. Once everyone has signed in with Google, you can turn passwords off.
- **Google for Platform admins?** Currently no. Admin access is the most sensitive; keep it on Cognito with MFA unless you decide otherwise.
- **Hosting for Intelligence:** Cloudflare (set up) or Vercel (the original plan). Either works; pick one to avoid running both.
- **Data licence:** which provider(s) to contract for market listings, and who pays.
- **Prices and billing:** subscription prices and whether to automate payments.
- **Next.js upgrade** and desktop code-signing, as noted in section 3.

8. Where things are in the repository

Document	Use
`docs/SINGLE_SIGN_IN.md`	Google single sign-in design and owner steps.
`docs/INTELLIGENCE_PRODUCTION.md`	Intelligence provisioning, environment, release, rollback, Cloudflare option.
`docs/LIVE_READINESS.md`	Step-by-step live runbook (migrations, secrets, deploy, acceptance).
`docs/STAFF_IDENTITY_SETUP.md`	Cognito stack and secrets mapping.
`docs/inventory-intake.md`, `docs/dealer-data-api.md`, `docs/PROVIDER_SYNC.md`	Inventory import, sourcing data and licensed feeds.
`docs/RELEASE_TRACKER.md`	Evidence and what remains unproven.

This guide describes the repository as of the commit that contains it. Items marked Done are implemented and locally tested; none has been verified against live Google, AWS or Cloudflare services.